TASK 6

Measuring Computational Efficiency

A 2D comparison of Python loops, NumPy vectorization, and compiled C

Atomic Toy Model Project

Benchmark run: 5 November 2026 | Reference execution environment: Linux, Python 3.13.5, NumPy 2.3.5, GCC 14.2.0

Purpose	Benchmark design
Determine how implementation choice affects pair-force runtime as particle count increases.	Same 2D positions and force parameters for three implementations; N = 50, 200, 500, 1000; five timing samples per method and N.

Executive summary

This experiment compares the pair-force calculation in three forms: a pure-Python double loop, the full NumPy broadcasting implementation used by the team in task3_4.py, and a compiled C kernel adapted from the newly supplied pairforce.c. The C kernel was loaded into Python with ctypes. For every particle count, all three methods received the same deterministic float64 position array and used k = 15.0, r0 = 1.5, and rc = 1.5.

In this reference run, compiled C was fastest at all tested sizes. At N = 1000, its median time was 1.3958 ms per force call, compared with 61.5976 ms for NumPy and 121.2250 ms for the Python loop. This corresponds to speedups of 86.85x over Python and 44.13x over NumPy. These are reference-host measurements; the final project should use the CSV results obtained when the team runs the package on its own computer.

1. Objective and project requirements

Task 6 asks for an actual timing comparison of the pair-force calculation using (i) a Python double loop, (ii) a NumPy-vectorized implementation, and (iii) compiled C. The required particle counts are at least N = 50, 200, 500, and 1000. Each case must be timed at least five times, reported using the median wall-clock time, and compared using speedup relative to the Python loop.

2. Model and implementations

2.1 Force law and parameters

For particles i and j at positions r_i and r_j, define d_ij = r_i - r_j and r_ij = ||d_ij||. When 0 < r_ij <= rc, the pair force is:

$$F_{ij} = -k\,(r_{ij} - r_0)\,d_{ij} / (r_{ij} + \text{epsilon})$$

The small epsilon = 10^-12 in the denominator matches the unit-vector calculation in the uploaded task3_4.py. Pairs at zero separation and pairs beyond the cutoff are excluded.

Parameter	Value used	Reason
Dimension	2D	Matches task3_4.py position arrays with shape (N, 2).
k	15.0	Copied from task3_4.py.
r0	1.5	Copied from task3_4.py.
rc	1.5	Copied from task3_4.py.
dtype	float64	Required by the uploaded C kernel and its double pointers.
N	50, 200, 500, 1000	Required benchmark sizes.

2.2 The three implementations

- Python double loop: visits only unique pairs (j > i), calculates the pair force, and adds equal and opposite contributions to particles i and j.
- NumPy vectorized: follows the full pairwise broadcasting pattern in task3_4.py. It constructs displacement, distance, neighbor, and force arrays for all ordered pairs.
- Compiled C (-O2): adapts the newly uploaded pairforce.c function. It visits each unique pair once and uses Newton's third law to update both particles.

The Python and C versions both exploit pair symmetry. The NumPy version evaluates the full N x N pair array. Therefore the benchmark compares the actual implementations, including their algorithmic differences; it is not a language-only comparison with identical operation counts.

2.3 Consistent input positions

For each N, one seeded random array of shape (N, 2) is generated and reused without modification by all three methods. To avoid increasing particle density as N grows, the accessible square side is scaled by sqrt(N/12), preserving approximately the density of the N = 12, L = 10 reference configuration with a margin of 0.8. The box length is recorded in the timing CSV.

3. Benchmark protocol

- Compile the adapted C source as a shared library using GCC -O2. Compilation time is not included in the force-call timing.
- Perform a correctness check before timing. NumPy and C force arrays must agree with the Python-loop reference within rtol = 1e-8 and atol = 1e-8.
- Warm up each implementation and calibrate a batch size so each timing sample measures a short batch rather than a single extremely short call.
- Collect five independent samples per method and N. Rotate method order between samples to reduce systematic order bias. Report the median time per force call.
- Time the full call, including the Python-to-ctypes call overhead for C, but exclude compilation, random data generation, and CSV/plot file writing.

3.1 Speedup formula

$$\text{Speedup(method)} = \text{median time(Python loop)} / \text{median time(method)}$$

The Python double-loop implementation is the baseline and therefore has a speedup of 1.0x for every N.

3.2 Reference execution environment

Item	Recorded value
Operating system	Linux 6.18.57 x86_64 (glibc 2.41)
Python	3.13.5
NumPy	2.3.5
C compiler	GCC 14.2.0
Compiler optimization	-O2
Timing samples	5 per method per N

The processor model was not reported by the runtime environment. The timings below must therefore be treated as reference-host measurements, not as measurements from the student laptop.

4. Measured wall-clock times

N	Implementation	Median time (ms)	Speedup
50	Python	0.37776	1.00x
50	NumPy	0.16026	2.36x
50	C (-O2)	0.00960	39.34x
200	Python	5.24820	1.00x
200	NumPy	2.13655	2.46x
200	C (-O2)	0.06575	79.82x
500	Python	32.19230	1.00x
500	NumPy	18.32629	1.76x
500	C (-O2)	0.36843	87.38x
1000	Python	121.22502	1.00x
1000	NumPy	61.59756	1.97x
1000	C (-O2)	1.39580	86.85x

Table 1. Median time per force call. Each median is calculated from five independently measured timing samples.

5. Timing and speedup plots

The wall-time plot uses a logarithmic vertical axis so the difference between the slow Python loop and the compiled C kernel remains visible at every particle count.

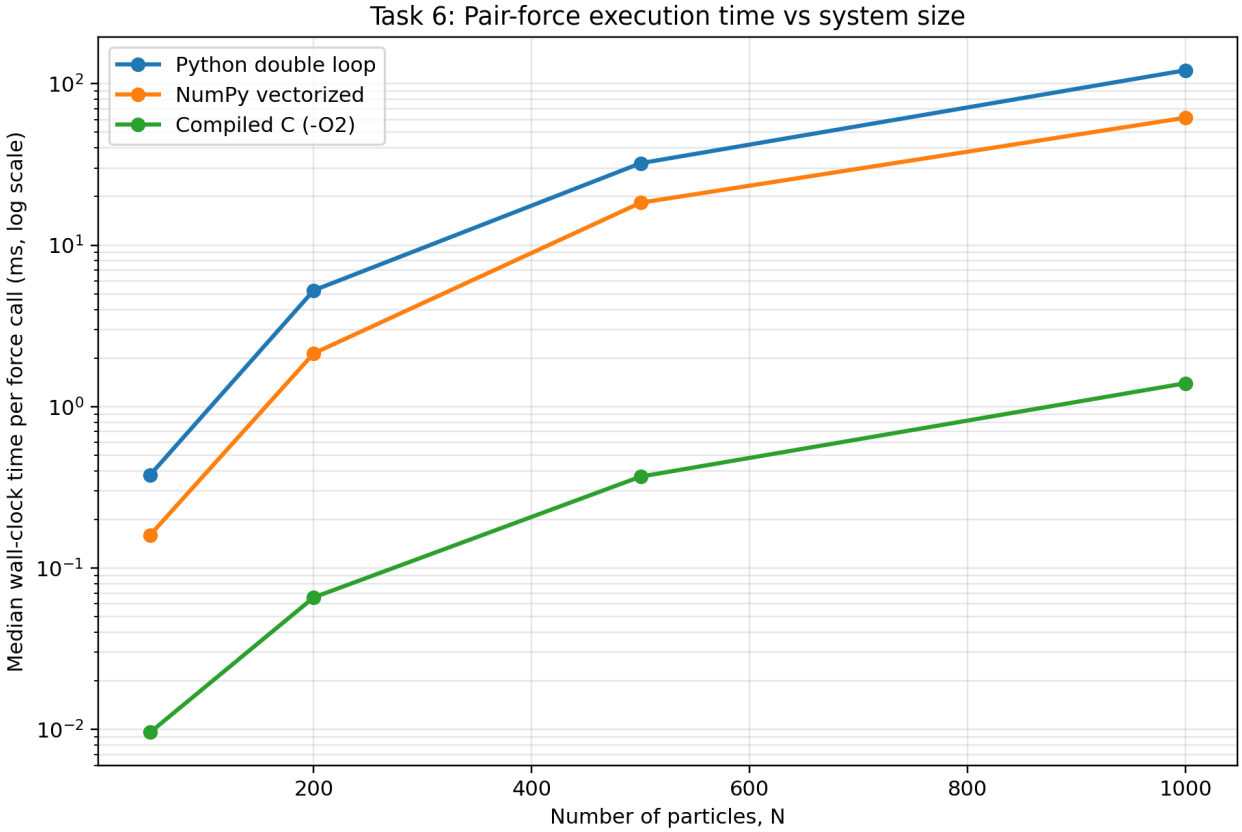


Figure 1. Median wall-clock time per force call versus particle count N. Lower is faster.

The speedup plot uses the pure-Python double loop as the 1.0x baseline. Values above 1 indicate an implementation faster than the baseline.

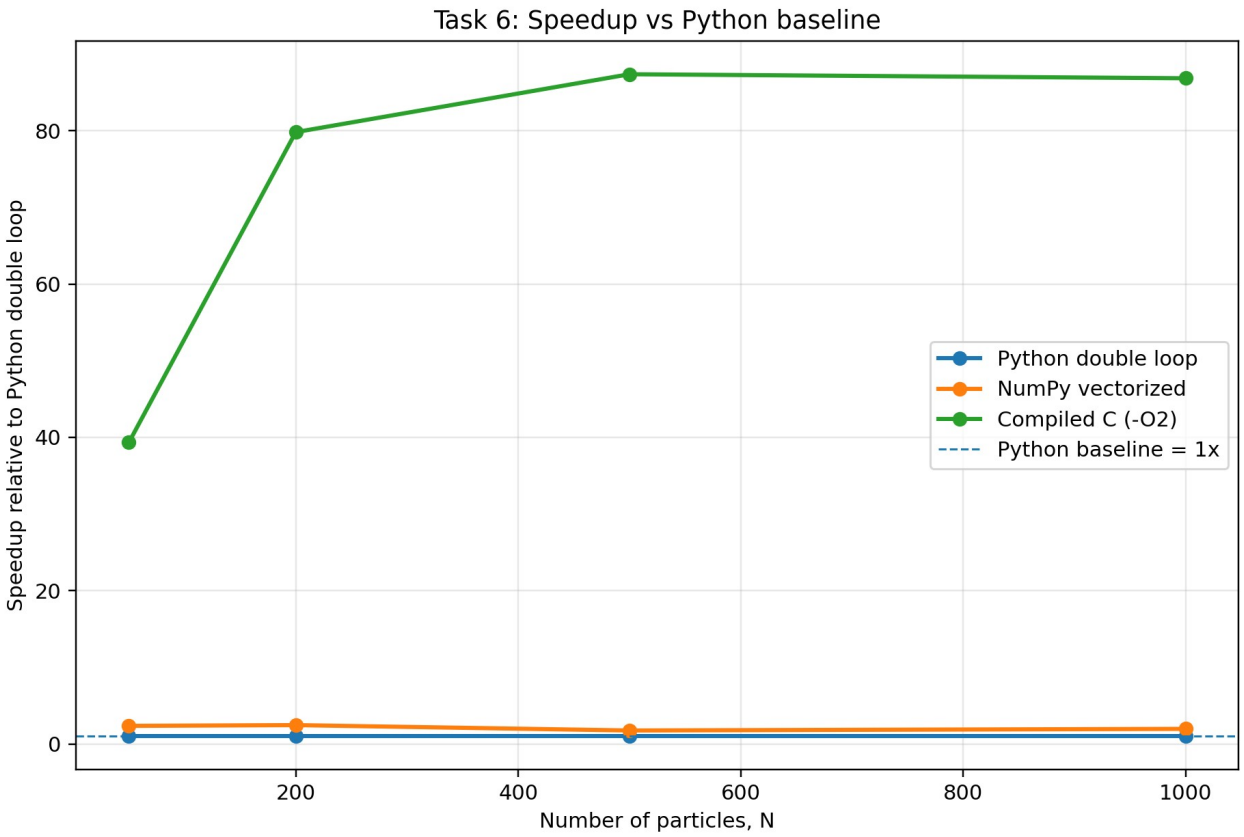


Figure 2. Speedup relative to the Python double-loop implementation.

6. Numerical verification

Timing is meaningful only if the methods calculate the same force field. Before timing, the force arrays were compared for every N. The net internal force was also checked using the Python reference output.

N	Max |Python - NumPy|	Max |Python - C|	||sum F||	Status
50	3.55e-15	0.00e+00	3.20e-15	PASS
200	3.55e-15	0.00e+00	2.05e-14	PASS
500	7.11e-15	0.00e+00	4.91e-14	PASS
1000	7.11e-15	0.00e+00	1.93e-13	PASS

Table 2. Force-equivalence and Newton-third-law checks. Small non-zero net-force residuals arise from floating-point summation order.

7. Interpretation and computational bottlenecks

7.1 Main observations

- Compiled C is the fastest implementation at every tested N. At N = 1000, it is 86.85x faster than the Python loop and 44.13x faster than NumPy.
- NumPy is faster than the Python loop at every tested size, with speedups between 1.76x and 2.46x. Its advantage is reduced by the allocation and processing of multiple N x N intermediate arrays.
- The Python and C loops have interpreted-language overhead and compiled native execution, respectively. C also avoids creating large pairwise NumPy temporaries and visits each unordered pair once.

7.2 Complexity and bottlenecks

All three methods use pairwise comparison and therefore have O(N^2) pair-search complexity. The Python implementation is limited primarily by interpreted nested-loop execution and scalar operations. NumPy moves arithmetic into compiled array kernels, but incurs memory traffic and temporary-array allocation for displacement, distance, neighbor, unit-vector, and force arrays. The C kernel uses contiguous arrays, native loops, and pair symmetry, reducing both interpreter overhead and intermediate memory allocation.

The timing trend broadly follows quadratic growth as N increases. Small timing irregularities are expected from CPU scheduling, cache effects, timer resolution, and memory allocation. The median of five samples reduces the influence of isolated timing outliers.

8. Conclusion

For the tested 2D force model and N values from 50 to 1000, the compiled C implementation provided the lowest median execution time. NumPy was consistently faster than the pure-Python loop, but slower than C for this workload. The results support using the C kernel when pair-force calculation speed is the priority, while NumPy remains convenient for readable, compact scientific code.

The comparison also shows that a performance gain is not caused by the programming language alone. The C kernel combines compiled execution with Newton's third-law pair symmetry and does not allocate full N x N displacement and force intermediates. These choices help explain its advantage over the full-broadcast NumPy implementation.

9. Reproduction on the project computer (Windows / VS Code)

- Install Python dependencies: python -m pip install numpy matplotlib
- Install GCC via MSYS2/MinGW-w64 and add the MinGW-w64 bin directory to PATH.
- Open a new VS Code terminal and verify: gcc --version
- Run: python task6_benchmark.py
- For more stable timing statistics, run: python task6_benchmark.py --samples 7
- Use the generated task6_results/task6_timing_summary.csv, task6_timing_samples.csv, task6_verification.csv, and plots in the final report.

10. Implementation correction and limitations

The uploaded team C source uses the function signature compute_pair_forces(N, pos, F, rc, r0, k). The earlier standalone Task 6 wrapper passed the final three values in a different order. That earlier C timing data is invalid and has been replaced by this revised benchmark. The revised wrapper passes rc, r0, and k in the correct order and verifies the C force array against the Python-loop reference before collecting timing samples.

The shared-library C source in this package is adapted from the uploaded pairforce.c. It retains the function interface and pairwise calculation, removes the standalone four-particle main() from the library build, and adds the same 1e-12 denominator offset as task3_4.py. The original uploaded C file is preserved separately in the package for comparison.

The parameter values follow the current uploaded task3_4.py (k = 15, r0 = 1.5, rc = 1.5), not the original handout values (r0 = 0.8, rc = 2.0). The team should resolve this model-setting difference consistently across the project before final submission. Timings depend on hardware and software versions; the team must rerun this benchmark on its own laptop and use those local measurements as the final experimental evidence.

Files generated by the benchmark

File	Contents
task6_timing_summary.csv	Median, minimum, maximum runtime and speedup for each N and method.
task6_timing_samples.csv	Five individual per-call timing samples for every method and N.
task6_verification.csv	Force-array agreement and net internal-force residuals.
task6_wall_time_vs_N.png	Required wall-time versus N plot.
task6_speedup_vs_N.png	Speedup relative to the Python baseline.
task6_environment.txt	OS, Python, NumPy, compiler, flags and benchmark settings.